☕ JAVA OOPS (Object Oriented Programming System)

What is OOPS?

OOPS is a **programming approach** in which programs are designed using **classes and objects**. **Object-Oriented Programming System (OOPS)** is a programming paradigm that organizes a program around **objects and classes** rather than functions and logic. It is based on the concept of real-world entities by combining **data (variables)** and **behaviour (methods)** into a single unit called a class. OOPS focuses on principles such as **Encapsulation, Inheritance, Polymorphism, and Abstraction**, which help in improving code **reusability, security, modularity, and maintainability**. By using OOPS, complex software systems become easier to design, understand, extend, and manage.

👉 Why do we use OOPS?

- Code becomes **reusable**
- Programs are **easy to manage and maintain**
- Provides **security**
- Helps represent **real-world entities** easily

◆ Class & Object

📓 What is a Class?

A **class** is a **blueprint or template** used to create objects in object-oriented programming. It defines the **properties (variables)** and **behaviours (methods)** that the objects created from it will have. A class does not occupy memory by itself; memory is allocated only when an **object of the class is created**. In simple terms, a class represents a general concept, while objects represent its real-world instances.

IN WHATSAPP LANGUAGE----

What is Class---

Class ek **blueprint / template** hoti hai jiske according **object bante hain**. Isme object ke **data (variables)** aur **kaam (methods)** define hote hain. Class khud memory nahi leti, memory tab milti hai jab uska **object create** hota hai. Simple words me, class ek idea hai aur object uska real use 👆

Example: Car is a class

📓 What is an Object?

An **object** is a **real-world instance of a class**. It represents a real entity that has its own **state (data/variables)** and **behaviour (methods)**. An object is created using a class and **occupies memory**,

allowing the program to access and use the data and methods defined in that class. In simple words, an object is the actual thing created from a class that performs actions in a program.

IN WHATSAPP LANGUAGE---

Object, **class ka real instance** hota hai. Jab hum class se koi cheez banate hain to use **object** kehte hain. Object ke paas apna **data (variables)** aur **kaam (methods)** hote hain aur **yeh memory leta hai**. Simple language me, class idea hoti hai aur object us idea ka real use ☺

Example: BMW and Audi are objects of the Car class

💻 Code Example:

```
class Student {
    int roll;
    String name;

    void display() {
        System.out.println(roll + " " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.roll = 1;
        s1.name = "Aparana";
        s1.display();
    }
}
```

◆ Four Pillars of OOPS

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

1️⃣ Encapsulation

📘 What is Encapsulation?

Encapsulation is an object-oriented programming concept in which **data (variables)** and **methods (functions)** are wrapped together into a single unit called a class. It restricts direct access to the data by making variables private and provides controlled access through public getter and setter methods. Encapsulation helps in improving **data security**, **code maintainability**, and prevents unwanted modification of data.

IN WHATSAPP LANGUAGE ---

Encapsulation ek OOPS concept hai jisme **data (variables)** aur **methods (functions)** ko **ek hi class ke andar bind** kar diya jata hai aur data ko **direct access se protect** kiya jata hai. Isme hum data ko `private` bana dete hain aur use access karne ke liye **public getter aur setter methods** use karte hain. Encapsulation se **security badhti hai**, code **easy to maintain** hota hai aur unwanted changes se data safe rehta hai.

🌍 Real-Life Example:

ATM machine – you cannot directly access cash; it is provided through a proper process.

💻 Code Example:

```
class Bank {
    private int balance = 10000;

    public int getBalance() {
        return balance;
    }

    public void setBalance(int amount) {
        balance = amount;
    }
}

public class Main {
    public static void main(String[] args) {
        Bank b = new Bank();
        System.out.println(b.getBalance());
        b.setBalance(15000);
        System.out.println(b.getBalance());
    }
}
```

2️⃣ Inheritance

📘 What is Inheritance?

Inheritance is an object-oriented programming concept in which one class (child/subclass) **acquires the properties and methods of another class** (parent/superclass). It promotes **code reusability** because common features are defined in the parent class and reused in the child class. Inheritance also helps in establishing a **relationship between classes**, making the program more organized and easier to maintain.
The `extends` keyword is used.

IN WHATSAPP LANGUAGE-----

Inheritance ek OOPS concept hai jisme ek class dusri class ke properties aur methods use karti hai**. Parent class ke features child class me aa jate hain, isse** code reuse hota hai **aur program** easy aur structured **ho jata hai. Java me inheritance ke liye `extends` keyword use hota hai**

🌍 Real-Life Example:

Father → Child (surname, habits, features)

💻 Code Example:

```java
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

📌 Types of Inheritance in Java

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance

⚠ Multiple inheritance is **not supported using classes**, but it is possible using **interfaces**.

3️⃣ Polymorphism

📘 What is Polymorphism?

Polymorphism **is an object-oriented programming concept that means** one method or object can have multiple forms**. In Java, the same method can perform different tasks either by changing the number or type of parameters (**method overloading**) or by redefining a parent class method in a child class (**method overriding**). Polymorphism increases** flexibility, reusability, and readability **of the code.**

IN WHATSAPP LANGUAGE-----

Polymorphism ek OOPS concept hai jiska matlab hota hai ek cheez ke multiple forms**. Java me ek hi method** alag-alag tarike se kaam kar sakta hai—**ya to parameters change karke (method overloading) ya parent class ke method ko child class me redefine karke (method overriding). Isse code** flexible, reusable aur easy to understand **ho jata hai** ☺

🔶 Types of Polymorphism:

1. Compile-time Polymorphism (Method Overloading)
2. Run-time Polymorphism (Method Overriding)

🔹 Method Overloading

Same method name with **different parameters**. **Method Overloading kya hota hai?** Method Overloading me **same class ke andar same method name** hota hai, lekin **parameters different** hote hain (number, type ya order). Java compile time par decide karta hai kaunsa method call hoga, isliye ise **compile-time polymorphism** bhi kehte hain. Isse code **easy aur reusable** ho jata hai 👆

```java
class MathOp {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args) {
        MathOp m = new MathOp();
        System.out.println(m.add(2, 3));
        System.out.println(m.add(2, 3, 4));
    }
}
```

◆ Method Overriding

Redefining a **parent class method** inside a child class.

Method Overriding is a concept in Java where a **child class provides its own implementation of a method that is already defined in the parent class**. The method in the child class must have the **same name, return type, and parameters** as in the parent. Overriding is used to achieve **run-time polymorphism** and allows a child class to **customize or extend the behaviour** of the parent class method.

```java
class Parent {
    void show() {
        System.out.println("Parent show");
    }
}

class Child extends Parent {
    void show() {
        System.out.println("Child show");
    }
}

public class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        p.show();
    }
}
```

4 Abstraction

📓 What is Abstraction?

Abstraction **is an object-oriented programming concept that** hides the implementation details **of a class and** shows only the essential features **to the user. It helps in** reducing complexity **and allows the programmer to focus on** what an object does rather than how it does it**. In Java, abstraction can be achieved using** abstract classes **and** interfaces**.**

IN WHATSAPP LANGUAGE-----

Abstraction ek OOPS concept hai jisme **implementation ka detail hide karke sirf important features dikhaye jate hain**. Isse code **simple aur easy to use** ho jata hai. Java me abstraction **abstract class ya interface** se achieve hoti hai ☺

🔷 Abstract Class Example

```
abstract class Vehicle {
    abstract void start();
}

class Bike extends Vehicle {
    void start() {
        System.out.println("Bike starts with key");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v = new Bike();
        v.start();
    }
}
```

◆ Interface Example

```java
interface Payment {
    void pay();
}

class UPI implements Payment {
    public void pay() {
        System.out.println("Payment via UPI");
    }
}

public class Main {
    public static void main(String[] args) {
        Payment p = new UPI();
        p.pay();
    }
}
```

◆ Difference: Abstract Class vs Interface

Abstract Class	Interface
Can have abstract & non-abstract methods	Only abstract methods
Does not support multiple inheritance	Supports multiple inheritance

---

◆ Advantages of OOPS

Object-Oriented Programming provides several important benefits which make Java powerful and widely used:

1. **Reusability** – Code can be reused using inheritance, reducing redundancy.
2. **Security** – Data is protected using encapsulation and access modifiers.
3. **Modularity** – Programs are divided into small, manageable classes.
4. **Maintainability** – Easy to modify, debug, and extend the program.
5. **Scalability** – Large applications can be built and expanded easily.
6. **Real-world mapping** – Objects closely represent real-world entities.

◆ Key OOPS Terminologies (Important Theory)

◈ Object

- Real-world entity
- Has **state** (variables), **behavior** (methods), and **identity**

◈ Class

- Logical group of objects
- Defines properties and behaviors

◈ Method

- A block of code that performs a specific task
- Improves code reusability

◈ Constructor

- Special method used to initialize objects
- Same name as class, no return type

```
class Demo {
    Demo() {
        System.out.println("Constructor called");
    }
}
```

◆ Access Modifiers (Theory)

Access modifiers define the **visibility** of variables, methods, and classes.

Modifier	Scope
public	Accessible everywhere
private	Within the same class
protected	Same package or subclass
default	Same package only

◆ this Keyword (Theory)

- Refers to the **current object** of the class
- Used to avoid confusion between instance variables and parameters

```java
class Student {
    int id;
    Student(int id) {
        this.id = id;
    }
}
```

◆ super Keyword (Theory)

- Refers to the **parent class object**
- Used to access parent class methods, variables, and constructors

```java
class A {
    int x = 10;
}
class B extends A {
    int x = 20;
    void show() {
        System.out.println(super.x);
    }
}
```

◆ Static Keyword (Theory)

- Belongs to the **class**, not object
- Memory efficient

```java
class Counter {
    static int count = 0;
    Counter() {
        count++;
        System.out.println(count);
    }
}
```

◆ Final Keyword (Theory)

- `final` variable → value cannot change
- `final` method → cannot be overridden
- `final` class → cannot be inherited

◆ Interview Practice Questions

1. Explain OOPS with real-life example.
2. Why Java is called an object-oriented language?
3. What is the difference between class and object?
4. What is constructor? Types of constructors?
5. What is the use of `this` and `super`?
6. Why multiple inheritance is not supported in Java?
7. Difference between static and non-static members